

From TS-SUF-2 to TS-SUF-4: Practical Security Enhancements for FROST2 Threshold Signatures

Syh-Yuan Tan, Will Wang, and Ryan Chow

Solv Protocol, Singapore

syhtansy@gmail.com, will@solv.finance, ryan@solv.finance

Abstract. Threshold signature schemes play a vital role in securing digital assets within blockchain and distributed systems. FROST2 stands out as a practical threshold Schnorr signature scheme, noted for its efficiency and compatibility with standard verification processes. However, under the one-more discrete logarithm assumption, with static corruption and centralized key generation settings, FROST2 has been shown by Bellare et al. (in CRYPTO 2022) to achieve only TS-SUF-2 security, which is a consequence of its vulnerability to TS-UF-3 attacks.

In this paper, we address this security limitation by presenting an enhanced variant of FROST2, namely, FROST2+ which achieves the TS-SUF-4 security level under the same computational assumptions as the original FROST2. FROST2+ strengthens FROST2 by integrating additional pre-processing token verifications that help mitigate TS-UF-3 and TS-UF-4 vulnerabilities while maintaining practical efficiency. We show that FROST2+ can achieve TS-SUF-4 security not only under the same conditions as the original FROST2 analysis, but also when initialized with a distributed key generation protocol such as PedPoP. Our benchmark using ZCash’s FROST library shows that the performance of FROST2+ is comparable to FROST2 and about 30% faster than FROST.

Keywords: Schnorr · Threshold Signature · FROST · TS-SUF · Provable Security

1 Introduction

The Schnorr signature scheme [27] has emerged as a cornerstone for blockchain applications due to its efficient signing and verification processes, compact signatures, and support for aggregation. In this context, Schnorr-based threshold signatures are particularly critical for threshold signing in blockchain systems [23,19,1,26], with FROST [21] establishing itself as a benchmark and currently undergoing IETF standardization [11]. Furthermore, NIST’s recent call for contributions on threshold multi-party schemes [8] has intensified research interest in FROST.

Research related to FROST can be categorized into two main areas: the development of new schemes that offer enhanced efficiency [14,7,5,22], and improvements in security [25,10,2,9,3,24,4]. The latter category can be further divided

into specific areas of security analysis focused on distributed key generation [10,14,5] and adaptive security [15,13,9,17,2,24,16,4]. Several works [7,5,12,13,17] delve deeper into the security notions relevant to threshold signature schemes.

The TS-(S)UF- i security framework [7,5] represents a recent effort to rigorously formalize the security of threshold signature schemes. This framework groups a message M along with its auxiliary signing information as a leader request (lr) and classifies unforgeability levels according to adversarial capabilities, ranging from the weakest (TS-UF-0, which resists less capable attackers) to the strongest (TS-UF-4, which resists more capable attackers). While the framework typically describes forgeries in terms of triviality, we outline the attackers' abilities at each level for clarity:

1. TS-UF-0: The threshold scheme can withstand an attacker who cannot obtain any partial signatures from honest signers for the target message M .
2. TS-UF-1: The threshold scheme can withstand an attacker at TS-UF-0 who also queries at most $t - c - 1$ partial signatures on M from honest signers.
3. TS-UF-2: The threshold scheme can withstand an attacker at TS-UF-1 who also queries leader requests (lr) on M and obtain at most $t - c - 1$ honest partial signatures per lr .
4. TS-UF-3: The threshold scheme can resist an attacker at TS-UF-2 who also corrupts pre-processing tokens (pp_i) for honest signers $i \in lr.S$ in the signing set, but not for all non-responding honest signers in the same set.
5. TS-UF-4: The threshold scheme can resist an attacker at TS-UF-3 who corrupts pre-processing tokens (pp_i) for all honest signers $i \in lr.S$ in the signing set, though it cannot obtain partial signatures from every honest signer in $lr.S$.

where the descriptions of TS-UF-3 and TS-UF-4 refer to the adversary capabilities in echo schemes [5,7,12] like FROST. In such schemes, an honest signer i only produces a partial signature if the leader request (lr) contains i 's honestly generated pre-processing token $pp_i \in PP_i$. The framework also introduces the concepts of strong unforgeability (TS-SUF- i), which ties a request lr to each unique signature, beyond just the message M .

Moreover, the framework provides valuable insights into the relationships among security levels. For instance, it demonstrates that achieving TS-UF-0 security implies TS-UF-1 security, risking a security loss of $\binom{n}{t-1}$ for total signers n and threshold t . Additionally, it shows that combining a threshold scheme achieving TS-UF-3 security with a standard digital signature scheme with existential unforgeability under chosen-message attacks (EUF-CMA) results in a threshold scheme with TS-UF-4 security.

Within this security framework, both FROST and FROST2 achieve TS-UF-0 (and thereby TS-UF-1) security in the distributed key generation (DKG) settings [21,14,5], which represents the baseline security level within the TS-UF- i hierarchy. Under the centralized key generation (CKG) setting, FROST achieves TS-SUF-3 security and can be further improved to attain the highest TS-SUF-4 security level by incorporating an additional digital signature scheme for each signer

[7,5,12]. However, this approach introduces additional computational overhead and implementation complexity. More critically for our work, FROST2 [14,7,5] has been shown to achieve only TS-SUF-2 security and is not qualified for this security upgrade.

The limitation of FROST2 to TS-SUF-2 security arises from a fundamental vulnerability to TS-UF-3 attacks, as demonstrated by Bellare et al. [7,5]. This attack is reminiscent of the rogue public key attack in multi-signature schemes, exploiting a weakness in FROST2’s pre-processing token mechanism to forge a Schnorr signature. Consequently, this enables a forgery under an unauthorized signer set without needing to know the partial signatures of all honest signers, thereby compromising TS-UF-3 security (see Appendix A.1).

Enhancements to FROST2 have been proposed, but none have achieved TS-SUF-3 security or higher. For instance, ROAST [25] introduces token aggregation and signature share verification, improving robustness while acknowledging security reduction to the original FROST2. Its successor, Olaf [10], combines ROAST with a simplified distributed key generation (DKG) protocol (SimplPedPoP) from FROST’s PedPoP, proving its TS-SUF-2 security. Recent theoretical results [13,16] indicate that FROST, FROST2, and FROST3 (or ROAST) are either unforgeable with full adaptive security in the algebraic group model (AGM), under the assumptions of Algebraic One-More Discrete Logarithm (AOMDL) and Low-Dimensional Vector Representation (LDVR); or adaptively secure with a maximum of $t/2$ corruptions, assuming AOMDL in the random oracle model (ROM).

FROST-Mask [9], which can be viewed as an extension of FROST2 with masking mechanisms applied to the pre-processing tokens and partial signatures, has been proven unforgeable with adaptive security under the ROM. Conversely, ms-FROST [2] and FaFROST [4] apply masking solely to the partial signature, proving to achieve TS-UF-4 and adp-TS-UF, respectively, with adaptive security under the weaker AGM.

In this work, we aim to address whether FROST2, which enhances efficiency compared to FROST and reduces communication rounds relative to other variants [14,3,24], can attain the same TS-SUF-3 or TS-SUF-4 security levels as FROST under ROM.

1.1 Our Contributions

We proposed an improved variant of FROST2 that achieve TS-SUF-4 security, the highest tier in Bellare et al.’s hierarchical framework, while maintaining the protocol’s efficiency and its two-round structure.

The proposed FROST2+ addresses the TS-UF-3 vulnerability of FROST2 by introducing a lightweight token authentication mechanism for pre-processing tokens. In contrast to Bellare et al.’s ATs[TS, DS] transformation [7,5], which requires each signer to generate and manage an additional digital signature key-pair, our approach authenticates the tokens using only the shared signing key and the secret exponents already present in the pre-processing tokens (R, S) . We establish the TS-SUF-4 security of FROST2+ under the one-more discrete

logarithm (OMDL) assumption within the ROM, allowing for static corruption and centralized key generation (CKG). In the process of proving TS-SUF-4 security, we develop the General Multi-Segment Forking Lemma, an extension of the General Forking Lemma [6], capable of handling protocols with segmented transcript structures and is of independent interest.

Subsequently, we show that our proposed schemes maintain their security levels when transitioning from the CKG setting to a distributed key generation (DKG) setting, specifically using PedPoP DKG as an instantiation. We also explain how to speed up the partial signing algorithm by pre-verifying the pre-processing tokens, which is not bind to the signing message. Moreover, similar to FROST’s offline pre-processing, our token verification mechanism facilitates per-signer batching across multiple signing sessions. This design enables a signer to generate and verify n tokens with a constant-size transcript, distinguishing it from some variants with masking [9], which only support batching across signers in a single session but become inefficient in multi-session scenarios due to per-token overhead. This design choice preserves the efficiency of FROST-style pre-processing, making our schemes particularly advantageous for high-throughput, leaderless environments where offline token preparation is critical.

Finally, we show that the proposed FROST2+, while offering stronger security, achieves better efficiency than FROST and matches the performance of FROST2, as detailed in our performance evaluation (Section 6).

Organization. This paper is structured as follows. In Section 2, we present the necessary mathematical tools and introduce our General Multi-Segment Forking Lemma, which is elaborated upon in Section 2.3. We then outline the proposed FROST2+ scheme in Section 3, followed by the security analysis in Section 4. Next, we discuss the promising extensions of our schemes in Section 5 and present the performance evaluation in Section 6. We conclude the paper in Section 7.

2 Preliminaries

We briefly recall the required mathematical tools in this section.

2.1 One-More Discrete Logarithm (OMDL) Assumption

Definition 1. Let $\mathbb{G}$ be a cyclic group of prime order p with generator g , and let CHALL and DLOG be oracles such that:

- CHALL() returns a random challenge $A = g^a$ for $a \xleftarrow{\$} \mathbb{Z}_p^*$
- DLOG(B) returns $b \in \mathbb{Z}_p^*$ such that $B = g^b$ for any $B \in \mathbb{G}$

The OMDL assumption states that for any probabilistic polynomial-time adversary $\mathcal{C}$ making at most $q - 1$ queries to DLOG, the advantage

$$\text{Adv}_{\mathbb{G}}^{\text{omdl}}(\mathcal{C}) = \Pr \left[\begin{array}{l} (A_1, \dots, A_q) \leftarrow \text{CHALL}(), \\ (a_1, \dots, a_q) \leftarrow \mathcal{C}^{\text{CHALL}, \text{DLOG}}(g) \\ \forall i : g^{a_i} = A_i \wedge \mathcal{C} \text{ never queried DLOG}(A_i) \end{array} \right]$$

is negligible.

2.2 Lagrange Coefficient

Let $f(\cdot)$ be a random $(t - 1)$ -degree polynomial and $f(x_i) = s_i$, the polynomial $f(\cdot)$ can be reconstructed by having the knowledge of t -pair values of (x_i, s_i) :

$$f(\cdot) = \sum_{x_i \in \mathcal{S}} s_i \Delta_{x_i, \mathcal{S}}(\cdot)$$

where $\mathcal{S} = \{x_1, x_2, \dots, x_t\}$.

Definition 2. The Lagrange coefficient $\Delta_{x_i, \mathcal{S}}$ is defined as:

$$\Delta_{x_i, \mathcal{S}}(Z) = \prod_{x_j \in \mathcal{S}, j \neq i} \frac{Z - x_j}{x_i - x_j}$$

where x_i and x_j are elements in the set $\mathcal{S}$.

2.3 General Multi-Segment Forking Lemma

We generalize the forking lemma of Bellare and Neven [6] (c.f. Appendix B) in Algorithm 1 to accommodate multiple disjoint segments of freshly sampled hash values interleaved with unchanged values, starting from a single fork index I . This variant supports reductions where an adversary's random oracle responses are re-randomized in non-contiguous blocks, enabling extraction at up to e designated positions within these segments (with $e = 1$ at I in the base case). This captures protocols with segmented transcript structures or multi-round challenges, such as multi-signer schemes.

Lemma 1 (General Multi-Segment Forking Lemma). *Let acc be the probability that $\mathcal{A}$ returns a non-zero primary index I_1 and valid disjoint segments in its first execution in GMultiSegFork . Let frk be the probability that $\text{GMultiSegFork}(x)$ outputs $(1, \sigma, \sigma')$. Then for $e = \sum_{j=1}^k \ell_j$ total resampled positions (where $\ell_j = \tilde{I}_j - I_j + 1$):*

$$\text{frk} \geq \text{acc} \left(\frac{\text{acc}}{q} - \frac{e}{|\mathbb{H}|} \right)$$

Alternatively,

$$\text{acc} \leq \sqrt{q \cdot \text{frk}} + \frac{qe}{|\mathbb{H}|}.$$

Proof. The proof follows the structure from Bellare and Neven's work [6], adapted to the multi-segment resampling with multiple extraction points. The key insight is that the Cauchy-Schwarz applications and averaging hold regardless of which subset of positions after $I_1 - 1$ are resampled, as long as the space sizes adjust accordingly. We use discrete counting for concreteness, assuming $\mathbb{H}$ and the random tapes R (set of possible ρ) are finite. Assume $|R|$ is the size of the random tape space (finite for PPT $\mathcal{A}$). Fixing an input x , we prove the bound for this x and then take the expectation over $x \leftarrow IG$ as in [6]. We begin by defining the set

Algorithm 1 General Multi-Segment Forking Algorithm $\text{GMultiSegFork}_{\mathcal{A}}(x)$

```

1: Pick coins  $\rho$  for  $\mathcal{A}$  at random
2:  $\{h_1, \dots, h_q\} \xleftarrow{\$} \mathbb{H}^q$ 
3:  $Q \leftarrow \{h_1, \dots, h_q\}$ 
4:  $(([I_1..I_1], \dots, [I_k..I_k]), \sigma) \leftarrow \mathcal{A}(x, Q; \rho)$  ▷ Outputs indices of segments
5: if  $I_1 = 0$  or segments overlap then
6:   return  $(0, \perp, \perp)$ 
7: end if
8: for each new segment  $j = 1$  to  $k$  do
9:    $\{h'_{I_j}, \dots, h'_{\tilde{I}_j}\} \xleftarrow{\$} \mathbb{H}^{\ell_j}$  where  $\ell_j = \tilde{I}_j - I_j + 1$ 
10: end for
11:  $Q' \leftarrow \{h_1, \dots, h_{I_1-1}\}$  ▷ Start with unchanged prefix up to  $I_1 - 1$ 
12: for  $j = 1$  to  $k$  do ▷ Append alternating new/old segments
13:    $Q' \leftarrow Q' \cup \{h'_{I_j}, \dots, h'_{\tilde{I}_j}\}$ 
14:   if  $j < k$  then
15:      $Q' \leftarrow Q' \cup \{h_{\tilde{I}_j+1}, \dots, h_{I_{j+1}-1}\}$ 
16:   end if
17: end for
18:  $Q' \leftarrow Q' \cup \{h_{\tilde{I}_k+1}, \dots, h_q\}$  ▷ Append remaining old hashes after last segment
19:  $(([I'_1..I'_1], \dots, [I'_k..I'_k]), \sigma') \leftarrow \mathcal{A}(x, Q'; \rho)$ 
20: if  $\forall j : (I'_j = I_j \wedge \tilde{I}'_j = \tilde{I}_j) \wedge \forall j \forall pos \in [I_j..I_j] : (h_{pos} \neq h'_{pos})$  then
21:   return  $(1, \sigma, \sigma')$ 
22: else
23:   return  $(0, \perp, \perp)$ 
24: end if

```

S as the collection of all pairs $(\{h_1, \dots, h_q\}, \rho)$ such that $\mathcal{A}(x, \{h_1, \dots, h_q\}; \rho)$ outputs some $I_1 \geq 1$ and valid disjoint segments. The acceptance probability satisfies $\text{acc} = |S|/(|\mathbb{H}|^q \cdot |R|)$. For each $j \in \{1, \dots, q\}$, let $S_j \subseteq S$ denote the subset where the primary index is exactly $I_1 = j$. Then, $\text{acc} = \sum_{j=1}^q \text{acc}_j$, where $\text{acc}_j = |S_j|/(|\mathbb{H}|^q \cdot |R|)$. The forking success probability decomposes as $\text{frk} = \sum_{j=1}^q \text{frk}_j$, where frk_j is the probability that the algorithm succeeds conditioned on the first run outputting primary index $I_1 = j$ (i.e., segments match exactly and $h_{pos} \neq h'_{pos}$ for all pos in all segments). We express

$$\text{frk}_j = \text{acc}_j \cdot \Pr[\text{segments match} \wedge (\forall pos : h_{pos} \neq h'_{pos}) \mid I_1 = j].$$

Since the second execution resamples only the designated new segments independently, this conditional probability is the average over all accepting first-run inputs in S_j . To bound the conditional, let A denote the event segments match exactly, and B denote $\exists pos : h_{pos} = h'_{pos}$ (at least one collision in resampled positions). Since the resamples are independent and each resampled position

collides with probability $1/|\mathbb{H}|$,

$$\begin{aligned}\Pr[A \wedge (\forall pos : h_{pos} \neq h'_{pos})] &= \Pr[A] - \Pr[A \wedge B] \\ &\geq \Pr[A] - \Pr[B], \\ &\geq \Pr[A] - e/|\mathbb{H}|,\end{aligned}\tag{1}$$

where $\Pr[B] \leq \sum_{i=1}^e 1/|\mathbb{H}| = e/|\mathbb{H}|$ by union bound and $e = \sum_{l=1}^k \ell_l$ is the total resampled hashes. Thus, the conditional is at least $\mathbf{E}[\Pr[A] \mid I_1 = j] - e/|\mathbb{H}|$, where the expectation is over first-run inputs in S_j . For a fixed j , partition each input $(\{h_1, \dots, h_q\}, \rho) \in S_j$ into fixed parts (the old prefix positions 1 to $I_1 - 1 = j - 1$, old segments between new ones, and ρ) and resampled parts (the new segments). Recall that $e = \sum_{l=1}^k \ell_l$ denotes the total number of resampled positions after $j - 1$. The space size for resampled parts is $|\mathbb{H}|^e$, and for fixed parts is $|\mathbb{H}|^{q-e} \cdot |R|$. Define $K = |\mathbb{H}|^{q-e} \cdot |R|$, so the total space size is $K \cdot |\mathbb{H}|^e = |\mathbb{H}|^q \cdot |R|$. For each fixed part (α, ρ) , where α encompasses all non-resampled hashes, let $n_{\alpha, \rho}$ be the number of assignments to the resampled positions such that the full input lies in S_j . Then, $|S_j| = \sum_{\alpha, \rho} n_{\alpha, \rho}$, and for a fixed (α, ρ) , $\Pr[A \mid (\alpha, \rho)] = n_{\alpha, \rho}/|\mathbb{H}|^e$. The expectation satisfies

$$\begin{aligned}\mathbf{E}[\Pr[A] \mid I_1 = j] &= \frac{1}{|S_j|} \sum_{\alpha, \rho} n_{\alpha, \rho} \cdot (n_{\alpha, \rho}/|\mathbb{H}|^e) \\ &= \left(\sum_{\alpha, \rho} n_{\alpha, \rho}^2 \right) / (|\mathbb{H}|^e \cdot |S_j|).\end{aligned}\tag{2}$$

Applying the Cauchy-Schwarz inequality to the vector $(n_{\alpha, \rho})$ and an all-1 vector over the nonempty (α, ρ) set, we have

$$\left(\sum_{\alpha, \rho} n_{\alpha, \rho}^2 \right) \cdot (\# \text{ of } (\alpha, \rho) \text{ with } n_{\alpha, \rho} \geq 1) \geq \left(\sum_{\alpha, \rho} n_{\alpha, \rho} \right)^2 = |S_j|^2 \tag{3}$$

where at most K distinct (α, ρ) values exist, yielding $\sum n_{\alpha, \rho}^2 \geq |S_j|^2/K$. Substituting the bound from (3) into the equation (2) gives

$$\left(\sum_{\alpha, \rho} n_{\alpha, \rho}^2 \right) / (|\mathbb{H}|^e \cdot |S_j|) \geq (|S_j|^2/K) / (|\mathbb{H}|^e \cdot |S_j|) = |S_j| / (K|\mathbb{H}|^e).$$

Since $|S_j| = \text{acc}_j \cdot |\mathbb{H}|^q \cdot |R| = \text{acc}_j \cdot K \cdot |\mathbb{H}|^e$, it follows that $|S_j| / (K|\mathbb{H}|^e) = \text{acc}_j$ and therefore,

$$\mathbf{E}[\Pr[A] \mid I_1 = j] \geq \text{acc}_j. \tag{4}$$

Combining the bound from (1) with the expectation from (4), we get $\text{frk}_j \geq \text{acc}_j \cdot (\text{acc}_j - e/|\mathbb{H}|)$. Summing over j , we obtain

$$\begin{aligned} \text{frk} &\geq \sum_j \text{acc}_j (\text{acc}_j - e/|\mathbb{H}|) \\ &= \sum_j \text{acc}_j^2 - (e/|\mathbb{H}|) \sum_j \text{acc}_j \\ &= \sum_j \text{acc}_j^2 - (e/|\mathbb{H}|) \text{acc}. \end{aligned} \tag{5}$$

By Cauchy-Schwarz again, we know

$$\left(\sum_j \text{acc}_j^2 \right) \cdot q \geq \left(\sum_j \text{acc}_j \right)^2 = \text{acc}^2,$$

and so $\sum_j \text{acc}_j^2 \geq \text{acc}^2/q$. Substituting this into (5), we reach the required result

$$\text{frk} \geq \text{acc}^2/q - (e/|\mathbb{H}|) \text{acc} = \text{acc} (\text{acc}/q - e/|\mathbb{H}|).$$

The alternative bound follows by solving the quadratic inequality as in [6].

2.4 Threshold Signature Scheme

To synchronize with Bellare et al.'s TS-(S)UF- i security framework [7,5], we adopt their scheme definition while making some adjustments, including the addition of a **Setup** algorithm and modifications to certain inputs for each algorithm. We define a non-interactive threshold signature scheme as $\text{TS} = \{\text{Setup}, \text{Kg}, \text{SPP}, \text{LPP}, \text{LR}, \text{PS}, \text{Agg}, \text{Vf}, \text{SVf}\}$, characterized as follows:

$\text{Setup}(1^\lambda) \rightarrow \text{par}$: The setup algorithm takes in a security parameter 1^λ and produces public parameters par , which are implicitly provided to all other algorithms.

$\text{Kg}(\text{par}, t, \text{ns}) \rightarrow \{\text{st}_i = (\tilde{X}, \text{aux}, \text{sk}_i)\}_{i=1}^{\text{ns}}$: The key generation algorithm takes in threshold t and total signers ns such that $t \leq \text{ns}$, initializes the state st_i for every signer $\text{id}_i \in \{\text{id}_1, \dots, \text{id}_{\text{ns}}\}$ with joint public key $\tilde{X}$, public auxiliary information aux , and an individual shared signing key sk_i .

$\text{SPP}(\text{st}_i) \rightarrow (pp, \text{st}_i)$: The signing pre-processing algorithm takes as input the state st_i of signer id_i . It outputs a pre-processing token pp and updates the signer's state.

$\text{LPP}(\text{id}_i, pp, \text{st}_0) \rightarrow \text{st}_0$: The leader pre-processing algorithm takes the pre-processing token pp from signer id_i and the leader's state st_0 , updating st_0 to incorporate

the token.

$\text{LR}(M, \mathcal{S}, \text{st}_0) \rightarrow (lr, \text{st}_0)$: This leader signing-request algorithm generates a leader request lr based on a message M , a signing set $\mathcal{S}$ with at least t signers, and the leader's state st_0 .

$\text{PS}(lr, \text{st}_i) \rightarrow (psig_i, \text{st}_i)$: The signer partial-signing algorithm takes the leader request lr , the state st_i of a signer id_i , generating a partial signature $psig_i$ and updating state st_i .

$\text{Agg}(lr, \{psig_i\}_{\text{id}_i \in lr.\mathcal{S}}, \text{st}_0) \rightarrow (\sigma, \text{st}_0)$: The leader partial-signature aggregation algorithm aggregates the partial signatures received from the signing set $\mathcal{S}$ into a final signature σ for the message M while updating the leader's state st_0 .

$\text{Vf}(\tilde{X}, M, \sigma) \rightarrow 0 \text{ or } 1$: The verification algorithm checks the validity of the signature σ for the message M using the public key $\tilde{X}$, returning 1 if valid and 0 otherwise.

$\text{SVf}(\tilde{X}, lr, \sigma) \rightarrow 0 \text{ or } 1$: The strong verification algorithm checks the validity of the signature σ for the leader request lr using the public key $\tilde{X}$, returning 1 if valid and 0 otherwise.

2.5 Security Model

Table 1. Trivial forgery predicates for TS-(S)UF- i [7,5] with Cremers et al.'s recommendation [12].

Predicate	Definition
$\text{tf}_0(M)$	$S_1(M) = \emptyset$
$\text{tf}_1(M)$	$ S_1(M) \geq t - CS $
$\text{tf}_2(lr)$	$ S_2(lr) \geq t - CS $
$\text{tf}_3(lr)$	$\text{tf}_2(lr) \wedge S_2(lr) = S_3(lr)$
$\text{tf}_4(lr)$	$\text{tf}_2(lr) \wedge S_2(lr) = S_3(lr) = S_4(lr)$
$\text{tsf}_2(lr, \tilde{X}, \sigma)$	$\text{tf}_2(lr) \wedge \text{SVf}(\tilde{X}, lr, \sigma)$
$\text{tsf}_3(lr, \tilde{X}, \sigma)$	$\text{tf}_3(lr) \wedge \text{SVf}(\tilde{X}, lr, \sigma)$
$\text{tsf}_4(lr, \tilde{X}, \sigma)$	$\text{tf}_4(lr) \wedge \text{SVf}(\tilde{X}, lr, \sigma)$

To address a shortcoming in the original hierarchy of Bellare et al.'s hierarchy framework [5] (where TS-(S)UF-4 does not imply TS-(S)UF-3, and vice versa, due to incomparable trivial forgery conditions), instead of referring to their updated version [7], we adopt the strengthened trivial forgery condition in Table 1 for tf_4 as recommended by Cremers et al. [12]. Specifically, this modifies the trivial forgery condition tf_4 (or tsf_4 for strong variants) from requiring $S_2(lr) = S_4(lr)$

Games $\mathbf{G}_{\text{TS}}^{\text{ts-uf-}i}$ ($i = 0, 1, 2, 3, 4$) and $\mathbf{G}_{\text{TS}}^{\text{ts-suf-}i}$ ($i = 2, 3, 4$)

```

INIT( $CS$ ):
1: Require:  $CS \subseteq \{\text{id}_1, \dots, \text{id}_{\text{ns}}\}$  and  $|CS| < t$  ▷ Set of corrupted signers
2:  $h \xleftarrow{\$} \text{TS.HF}; (\tilde{X}, aux, sk_1, \dots, sk_{\text{ns}}) \xleftarrow{\$} \text{Kg}$  ▷ Set of hash functions TS.HF
3:  $HS \leftarrow \{\text{id}_1, \dots, \text{id}_{\text{ns}}\} \setminus CS$  ▷ Set of honest signers
4: for  $\text{id}_i \in HS$  do
5:    $st_i.sk \leftarrow x_i; st_i.\tilde{X} \leftarrow \tilde{X}; st_i.aux \leftarrow aux$ 
6: end for
7: return  $\tilde{X}, aux, \{x_i\}_{\text{id}_i \in CS}$ 

PPO( $\text{id}_i$ ):
8: Require:  $\text{id}_i \in HS$ 
9:  $(pp, st_i) \xleftarrow{\$} \text{SPP}(st_i); PP_i \leftarrow PP_i \cup \{pp\}$ 
10: return  $pp$ 

PSIGNO( $\text{id}_i, lr$ ):
11: Require:  $lr.S \subseteq \{\text{id}_1, \dots, \text{id}_{\text{ns}}\}, M = lr.msg \in \{0, 1\}^*$  and  $\text{id}_i \in HS$ 
12:  $(psig, st_i) \xleftarrow{\$} \text{PS}(lr, st_i); L \leftarrow L \cup \{lr\}$ 
13: if  $psig \neq \perp$  then
14:    $S_1(M) \leftarrow S_1(M) \cup \{\text{id}_i\}; S_2(lr) \leftarrow S_2(lr) \cup \{\text{id}_i\}$ 
15: end if
16: return  $psig$ 

RO[h]( $x$ ):
17: return  $h(x)$ 

FIN( $M, \sigma$ ):
18: for all  $lr \in L$  do
19:    $S_3(lr) \leftarrow \{\text{id}_i \in HS \cap lr.S : lr.PP(\text{id}_i) \in PP_i\}; S_4(lr) \leftarrow HS \cap lr.S$ 
20: end for
21: if  $\neg \forall f(\tilde{X}, M, \sigma)$  then
22:   return false;
23: end if
24: if Game  $\mathbf{G}_{\text{TS}}^{\text{ts-uf-}i}$  for  $i = 0, 1$  then
25:   return  $\neg \text{tf}_i(M)$ 
26: end if
27: if Game  $\mathbf{G}_{\text{TS}}^{\text{ts-uf-}i}$  for  $i = 2, 3, 4$  then
28:   return  $\neg \exists lr(lr.msg = M \wedge \text{tf}_i(lr))$ 
29: end if
30: if Game  $\mathbf{G}_{\text{TS}}^{\text{ts-suf-}i}$  then
31:   return  $\neg \exists lr(lr.msg = M \wedge \text{tsf}_i(lr, \tilde{X}, \sigma))$ 
32: end if

```

Fig. 1. Security games defining TS-(S)UF- i [5,7,12].

to the stricter $S_2(lr) = S_3(lr) = S_4(lr)$, ensuring that all honest signers in $lr.S$ responded with honestly generated pre-processing tokens.

This approach, equivalent to Cremers et al.’s SiGu-4:aLRhPP level, restores linear downward implications across the hierarchy (e.g., TS-(S)UF-4 implies TS-(S)UF-3) without weakening the lower levels as done in Bellare et al.’s revised version [7]. We prefer this strengthened TS-(S)UF- i model as shown in Figure 1 over Cremers et al.’s full framework [12] because it is simpler and not overkill for analyzing schemes like FROST2 under static corruption and centralized key generation.

Definition 3. Let $X \in \{\text{suf}, \text{uf}\}$ and $i \in \{0, 1, 2, 3, 4\}$. A TS scheme is secure if there is no polynomial time algorithm $\mathcal{F}$ that can win the game $\mathbf{G}_{\text{TS}}^{\text{ts-}X-i}$ in time t with non-negligible advantage:

$$\text{Adv}_{\text{TS}}^{\text{ts-}X-i}(\mathcal{F}) = \Pr[\mathbf{G}_{\text{TS}}^{\text{ts-}X-i}(\mathcal{F}) = \text{true}].$$

3 Our Constructions

In this section we present FROST2+ (shown in Figure 2) as a TS-SUF-4 variant of FROST2 from Bellare et al. [5]. Different from Bellare et al.’s ATS[TS, DS] transformation [5], which requires each signer to manage an extra digital signature key-pair to authenticate the pre-processing token (R, S) , our construction reuses only the existing signing key share and the secret exponents already present in the token. This yields a short authentication transcript while still supporting efficient batch verification (as discussed in Section 5.1).

4 Security Analysis

Our security proof adapts the security framework of Bellare et al. [7,5] with strengthened TS-(S)UF- i suggested by Cremers et al. [12] (c.f. Section 2.5), reducing unforgeability to the OMDL assumption under the random oracle model and static corruption setting.

Theorem 1. *Our proposed FROST2+ signature scheme with ns signers is TS-SUF-4 secure in the random oracle model based on the assumption of one more discrete logarithm (OMDL) such that if there exists an algorithm $\mathcal{F}$ that breaks the TS-SUF-4 security with advantage*

$$\text{Adv}_{\text{FROST2+}}^{\text{ts-suf-4}}(\mathcal{F}) \leq \sqrt{q \text{Adv}_{\mathbb{G}}^{\text{omdl}}(\mathcal{C})} + \frac{q^2}{p} + \frac{q}{\sqrt{p}}$$

in time $t_{\mathcal{F}}$, then there exists an algorithm $\mathcal{C}$ that breaks OMDL in time $t_{\mathcal{C}} \leq t_{\mathcal{F}} + T(q)$ where $T(q)$ are time parameterized by $q = q_s + q_h + \text{ns} + 2$ the number of signing and hashing queries made.

Proof. We show that if there is a forger $\mathcal{F}$ that can forge a FROST2+ signature to win in the game $\mathbf{G}_{\text{FROST2+}}^{\text{ts-suf-4}}$, then we can build an algorithm $\mathcal{C}$ to break the OMDL

Setup (1^λ) $(\mathbb{G}, p, g) \leftarrow \text{GrGen}(1^\lambda)$ $H_{\text{pp}}, H_{\text{non}}, H_{\text{sig}} : \{0, 1\}^* \rightarrow \mathbb{Z}_p^*$ $\text{par} \leftarrow ((\mathbb{G}, p, g), H_{\text{pp}}, H_{\text{non}}, H_{\text{sig}})$ return par	CompPar ($\tilde{X}, lr$) $M \leftarrow lr.\text{msg}$ $d \leftarrow H_{\text{non}}(\tilde{X}, lr)$ $(R_i, S_i) \leftarrow lr.\text{PP}(\text{id}_i)$ $\tilde{R} \leftarrow \prod_{\text{id}_i \in lr.S} R_i (\prod_{\text{id}_i \in lr.S} S_i)^d$ $c \leftarrow H_{\text{sig}}(\tilde{X}, M, \tilde{R})$ Return $(\tilde{R}, c, d)$
Kg (par, t, ns) : For $i = 0$ to $t - 1$, do $a_i \in \mathbb{Z}_p^*$ For $i = 1$ to ns , do $\text{sk}_i \leftarrow \sum_{j=0}^{t-1} a_j \times \text{id}_i^j; X_i \leftarrow g^{\text{sk}_i}$ $\tilde{X} = g^{a_0}; \text{aux} \leftarrow (\{X_i\}_{1 \leq i \leq \text{ns}})$ For $i = 1$ to ns , do $\text{st}_i \leftarrow (\tilde{X}, \text{aux}, \text{sk}_i)$ Return $\{\text{st}_i\}_{1 \leq i \leq \text{ns}}$	PS (lr, st_i) $pp_i \leftarrow lr.\text{PP}(\text{id}_i)$ If $\text{st}_i.\text{mapPP}(pp_i) = \perp$, return $(\perp, \text{st}_i)$ $\tilde{X}, \text{aux} \leftarrow \text{st}_i$ For $\text{id}_i \in lr.S$, do $(R_i, S_i, u_i, V_i) \leftarrow lr.\text{PP}(\text{id}_i)$ $e' = H_{\text{pp}}(\tilde{X}, X_i, R_i, S_i, V_i)$ If $g^{u_i} \neq V_i X_i^{e'} R_i^{e'^2} S_i^{e'^3}$, return $\perp$ Else $lr.\text{PP}(\text{id}_i) \leftarrow (R_i, S_i)$ $(r_i, s_i) \leftarrow \text{st}_i.\text{mapPP}(pp_i)$ $\text{st}_i.\text{mapPP}(pp_i) \leftarrow \perp$ $(\tilde{R}, c, d) \leftarrow \text{CompPar}(\text{st}_i.\tilde{X}, lr)$ $z_i \leftarrow r_i + d \cdot s_i + c \cdot \Delta_{\text{id}_i, S}(0) \cdot \text{st}_i.\text{sk}$ $psig_i \leftarrow (\tilde{R}, z_i)$ Return $(psig_i, \text{st}_i)$
SPP (st_i) $r, s, v \xleftarrow{\$} \mathbb{Z}_p^*$ $e = H_{\text{pp}}(\tilde{X}, X_i, R = g^r, S = g^s, V = g^v)$ $u = v + \text{st}_i.\text{sk} \cdot e + r e^2 + s e^3$ $pp \leftarrow (R, S, u, V)$ $\text{st}_i.\text{mapPP}(pp) \leftarrow (r, s, v)$ return (pp, st_i)	Agg ($\{psig_i\}_{\text{id}_i \in lr.S}, \text{st}_0$) $\tilde{R} \leftarrow \perp; z \leftarrow 0$ For $\text{id}_i \in S$, do If $\tilde{R} = \perp$, $\tilde{R} \leftarrow psig_i.\tilde{R}$ If $\tilde{R} \neq psig_i.\tilde{R}$, return $(\perp, \text{st}_0)$ $z \leftarrow z + psig_i.z$ $\sigma \leftarrow (\tilde{R}, z)$ Return (σ, st_0)
LPP ($\text{id}_i, pp, \text{st}_0$): $\text{st}_0.\text{currPP}_i \leftarrow \text{st}_0.\text{currPP}_i \cup \{pp\}$ return st_0	SVf ($\tilde{X}, lr, \sigma$) $(\tilde{R}, z) \leftarrow \sigma$ $(\tilde{R}', c', d) \leftarrow \text{CompPar}(\tilde{X}, lr)$ return $(\tilde{R} = \tilde{R}') \wedge (g^z = \tilde{R} \tilde{X}^{c'})$
LR (M, S, st_0) If $\exists \text{id}_i \in S : \text{st}_0.\text{curPP}(\text{id}_i) = \emptyset$, return $(\perp, \text{st}_0)$ $lr.\text{msg} \leftarrow M; lr.S \leftarrow S$ For $\text{id}_i \in S$, do Pick pp_i from $\text{st}_0.\text{curPP}(\text{id}_i)$ $lr.\text{PP}(\text{id}_i) \leftarrow pp_i$ $\text{st}_0.\text{curPP}(\text{id}_i) \leftarrow \text{st}_0.\text{curPP}(\text{id}_i) \setminus \{pp_i\}$ return (lr, st_0)	
Vf ($\tilde{X}, M, \sigma$) $(\tilde{R}, z) \leftarrow \sigma; c \leftarrow H_{\text{sig}}(\tilde{X}, M, \tilde{R})$ return $(g^z = \tilde{R} \tilde{X}^c)$	

Fig. 2. The FROST2 and FROST2+ signature schemes where FROST2 contains all but the solid box. GrGen generates cyclic group parameter in which the discrete logarithm assumption holds, st is state, $\text{ns} < p - 1$ is the total signers, and t is the signing threshold. The function `mapPP` records dlogs of tokens pp , `currPP` is the current pp collection, and `mapPP` returns dlogs of a pp .

assumption with the help of $\mathcal{F}$. To synchronize our proof with the General Multi-Segment Forking Lemma (cf. Section 2.3), we assume the existence of a single random oracle RO throughout the security proof, with its queries partitioned into three distinct categories: H_{pp} (pre-processing hashes), H_{non} (nonce hashes), and H_{sig} (signature hashes). Let $q_s = q_{s1} + q_{s2}$, the maximum allowed oracle queries throughout the game are denoted as $q = q_s + q_h + ns + 2$, which comprises q_{s1} queries through PPO, q_{s2} queries through PSIGNO, q_h queries through RO, and the $ns + 2$ hash queries reserved for a forgery.

Before the game $\mathbf{G}_{\text{FROST2+}}^{\text{ts-suf-4}}$ starts, $\mathcal{C}$ announces the signer set $\{\text{id}_1, \dots, \text{id}_{ns}\}$ and $\mathcal{F}$ decides the corrupted subset CS . $\mathcal{C}$ initializes hash lists $HL_{pp}, HL_{non}, HL_{sig}$ and a signing list L with empty strings $\emptyset$. $\mathcal{C}$ then queries $A_0 = g^{a_0}, \{A_j = g^{a_j}\}_{j=1}^k \leftarrow \text{CHALL}$ where $k = |CS| < t$ is the number of corrupted signers and samples $\{a_j\}_{j=k+1}^{t-1} \xleftarrow{\$} \mathbb{Z}_p^*$ to implicitly define $f(Z) = \sum_{j=0}^{t-1} a_j Z^j$. For every signer id_i , $\mathcal{C}$ computes their shared public key as $X_i = \prod_{j=0}^{t-1} A_j^{\text{id}_i^j}$ and sets $aux = (\{X_i\}_{1 \leq i \leq ns})$. For those $\text{id}_i \in CS$, $\mathcal{C}$ queries for $\text{sk}_i \leftarrow \text{DLOG}(X_i)$ and sets $\text{st}_i = (\tilde{X} = A_0, aux, \text{sk}_i)$. The initialization process issues $|CS| + 1$ CHALL queries and $|CS|$ DLOG queries in total. Subsequently, $\mathcal{C}$ simulates the game oracles as follows to interact with $\mathcal{F}$.

$\text{RO}[H_{pp}](M)$: If $(M, \cdot) \in HL_{pp}$, return e . Else, return $e \xleftarrow{\$} \mathbb{Z}_p^*$ and parse M as $(\tilde{X}, X_i, R, S, V)$. Update $HL_{pp} \leftarrow HL_{pp} \cup \{(M, e)\}$.

$\text{RO}[H_{non}](M)$: If $(M, \cdot) \in HL_{non}$, return d . Else, return $d \xleftarrow{\$} \mathbb{Z}_p^*$ and parse M as $(\tilde{X}, lr)$. Update $HL_{non} \leftarrow HL_{non} \cup \{(M, d)\}$.

$\text{RO}[H_{sig}](M)$: If $(M, \cdot) \in HL_{sig}$, return c . Else, return $c \xleftarrow{\$} \mathbb{Z}_p^*$ and parse M as $(\tilde{X}, lr.\text{msg}, \tilde{R})$. Update $HL_{sig} \leftarrow HL_{sig} \cup \{(M, c)\}$.

$\text{PPO}(\text{id}_i)$: $\mathcal{C}$ queries $\{R = g^r, V = g^v\} \leftarrow \text{CHALL}$ and selects $s \xleftarrow{\$} \mathbb{Z}_p^*$ to compute $S = g^s$. Let $M = (\tilde{X}, X_i, R, S, V)$. $\mathcal{C}$ queries for $e \leftarrow H_{pp}(M)$ and computes $u = v + \text{st}_i.\text{sk} \cdot e + re^2 + se^3 \leftarrow \text{DLOG}(V X_i^e R e^2 S e^3)$. $\mathcal{C}$ returns $pp = (R, S, u, V)$.

$\text{PSIGNO}(\text{id}_i, lr)$: When $\mathcal{F}$ queries for a partial signature $psig_i$ on lr , $\mathcal{C}$ rejects if $\text{id}_i \notin HS$ or $lr.S \not\subseteq \{\text{id}_1, \dots, \text{id}_{ns}\}$. Else, $\mathcal{C}$ retrieves $pp \leftarrow lr.PP(\text{id}_i)$ and obtains $e \leftarrow H_{pp}(\tilde{X}, X_i, R, S, V)$. $\mathcal{C}$ verifies whether $g^u = V X_i^e R e^2 S e^3$ holds and aborts if the verification fails. Next, $\mathcal{C}$ gets $d \leftarrow H_{non}(\tilde{X}, lr)$ and calculates $\tilde{R} = \prod_{\text{id}_i \in S} R_i (\prod_{\text{id}_i \in S} S_i)^d$ to query $c \leftarrow H_{sig}(\tilde{X}, lr.\text{msg}, \tilde{R})$ and obtain $z_i = \text{DLOG}(R_i S_i^d X_i^{c \Delta_{\text{id}_i, S}^{(0)}})$. $\mathcal{C}$ updates $L \leftarrow L \cup \{lr\}$, $S_1(lr.\text{msg}) \leftarrow S_1 \cup \{\text{id}_i\}$ and $S_2(lr) \leftarrow S_2 \cup \{\text{id}_i\}$ before returning $psig_i = (\tilde{R}, z_i)$.

Finally, $\mathcal{F}$ outputs a valid forgery $(M = lr.\text{msg}, \sigma = (\tilde{R}, z = r + ds + c \cdot a_0))$ such that $\text{SVf}(\tilde{X}, lr, \sigma) = 1$, and wins the game $\mathbf{G}_{\text{FROST2+}}^{\text{ts-suf-4}}$. Let $S_4(lr) \leftarrow \{\{\text{id}_1, \dots, \text{id}_{ns}\} \setminus CS\} \cap lr.S$ be the set of honest signers who suppose to sign

lr . When $\mathcal{F}$ wins, by the definition of TS-SUF-4, the event below must have happened:

$$E : (S_2(lr) < t - |CS|) \vee (S_2(lr) \neq S_3(lr) \neq S_4(lr)).$$

Upon observing the forgery, the simulator $\mathcal{C}$ identifies an index I_1 as the earliest $\text{RO}[\text{H}_{\text{pp}}]$ query in HL_{pp} for the first honest signer whose transcript $(\tilde{X}, X_i, R, S, V)$ appears in the list, among honest signers $\text{id}_i \in lr.\mathcal{S} \setminus CS$. This transcript is recorded when $\text{RO}[\text{H}_{\text{pp}}]$ is invoked in any of the four cases: (1) both PPO and PSIGNO queried; (2) only PPO queried; (3) only PSIGNO queried; or (4) neither queried (i.e., $\text{RO}[\text{H}_{\text{pp}}]$ queried directly). $\mathcal{C}$ then rewinds $\mathcal{F}$ to the index I_1 , enabling a fork in the random oracle simulation from that point onward. Without loss of generality, $\mathcal{C}$ reuses the same OMDL challenges and tokens $(S = g^s)$ in the rewind PPO queries. $\mathcal{C}$ updates HL_{pp} to pair all such transcripts with a new hash value e' , leaves $((\tilde{X}, lr), d) \in \text{HL}_{\text{non}}$ unchanged, but amends $((\tilde{X}, M, \tilde{R}), c) \in \text{HL}_{\text{sig}}$ to $((\tilde{X}, M, \tilde{R}), c')$. By the General Multi-Segment Forking Lemma, $\mathcal{F}$ outputs another valid forgery $(M, \sigma' = (\tilde{R}, z' = r + ds + c' \cdot a_0))$ and wins $\mathbf{G}_{\text{FROST2+}}^{\text{ts-suf-4}}$ again.

Now, we analyze the probability of success for $\mathcal{F}$, that is, the event E occurs. Note that $\mathcal{C}$ perfectly simulates the game except on the occasion of a hash collision, which occurs with a probability of $\frac{q}{p}$. It remains to show that when E happens and the reset is successful, $\mathcal{C}$ makes 1 query fewer to the DLOG oracle than to the CHALL oracle and breaks the OMDL assumption. Recall that during:

- INIT: $|CS| + 1$ CHALL queries and $|CS|$ DLOG queries issued;
- PPO: 2 CHALL and 1 DLOG queries issued, occurred for q_{s1} times;
- PSIGNO: 1 DLOG query issued, occurred for q_{s2} times

where $q_{s2} \leq q_{s1}$. After resetting $\mathcal{F}$, $\mathcal{C}$ obtains two valid signatures (M, σ) and (M, σ') where $\mathcal{C}$ can extract the discrete logarithm of $\tilde{X} = A_0 = g^{a_0}$:

$$a_0 = \frac{z' - z}{c' - c}.$$

Note that $\tilde{X} = g^{a_0} = g^{f(0)}$ and $f(\cdot)$ is of degree $t - 1$. Since $\mathcal{C}$ knows coefficients $\{a_j\}_{j=k+1}^{t-1}$, it can project the signing key $\text{st}_i.\text{sk} = f(\text{id}_i)$ of a corrupted signer to $\text{st}_i.\text{sk}' = f'(\text{id}_i)$ where $f'(\cdot)$ is of degree $k = |CS|$ such that $k \leq t - 1$:

$$\begin{aligned} \text{st}_i.\text{sk}' &= \text{st}_i.\text{sk} - \sum_{j=k+1}^{t-1} a_j \text{id}_i^j \\ &= f(\text{id}_i) - \sum_{j=k+1}^{t-1} a_j \text{id}_i^j \\ &= \sum_{j=0}^k a_j \text{id}_i^j = f'(\text{id}_i) \end{aligned}$$

So, $\mathcal{C}$ has $k + 1$ points $(0, a_0), (\text{id}_1, f'(\text{id}_1)), \dots, (\text{id}_k, f'(\text{id}_k))$ to interpolate the polynomial $f'(Z) = \sum_{j=0}^k a_j Z^j$ where the coefficients $\{a_j\}_{j=1}^k$ are the solutions for the challenges $\{A_j\}_{j=1}^k \leftarrow \text{CHALL}$ and recovers $|CS|$ discrete logarithms.

Now, for all honest signers $\text{id}_i \in \{\mathcal{S} \setminus CS\}$, with knowledge of signing key share $(\text{st}_i.\text{sk} = f(\text{id}_i))$, token (s) and two different transcripts $((u = v + \text{sk}_i e + re^2 + se^3, V), (u' = v + \text{sk}_i e' + re'^2 + se'^3, V))$, $\mathcal{C}$ can solve the $2q_{s1}$ discrete logarithms challenges $(R = g^r, V = g^v)$ in PPO. Particularly, $\mathcal{C}$ computes:

$$r = \frac{(u' - \text{st}_i.\text{sk} \cdot e' - se'^3) - (u - \text{st}_i.\text{sk} \cdot e - se^3)}{e'^2 - e^2}$$

and for (R, V) which is also queried to PSIGNO, $\mathcal{C}$ calculates $v = u - \text{sk}_i e - re^2 - se^3$. Otherwise, $\mathcal{C}$ queries $v \leftarrow \text{DLOG}(V)$.

Therefore, $\mathcal{C}$ solves $|CS| + 2q_{s1} + 1$ discrete logarithm with $|CS| + 2q_{s1}$ queries to the DLOG oracle only. According to the General Multi-Segment Forking Lemma, $\mathcal{C}$ breaks the OMDL assumption with advantage:

$$\begin{aligned} \Pr[\mathcal{C} \text{ wins}] &= \Pr[\mathcal{F} \text{ wins}] - \Pr[\mathcal{C} \text{ aborts}] \\ \text{frk} &= \Pr[E] \left(\frac{\Pr[E]}{q} - \frac{q}{p} \right) - \frac{q}{p} \\ &\geq \text{acc} \left(\frac{\text{acc}}{q} - \frac{q}{p} \right) - \frac{q}{p} \\ &\geq \frac{\text{acc}^2}{q} - \frac{q \cdot \text{acc}}{p} - \frac{q}{p} \end{aligned}$$

where $q = q_s + q_h + |lr.\mathcal{S}| + 2$ are signing (PPO + PSIGNO) and hashing (RO) queries made with $|lr.\mathcal{S}| \leq \text{ns}$.

Let $\text{acc} = \text{Adv}_{\text{FROST}_{2+}}^{\text{ts-suf-4}}(\mathcal{F})$ and $\text{frk} = \text{Adv}_{\mathbb{G}}^{\text{omdl}}(\mathcal{C})$, rewriting the inequality in the standard form $\text{acc}^2 - a \cdot \text{acc} - b \leq 0$ and solving it using the quadratic formula $\text{acc} = \frac{a \pm \sqrt{a^2 + 4b}}{2}$ for positive root gives:

$$\begin{aligned} \text{acc} &\leq \frac{q^2 + \sqrt{q^4 + 4q^2p + 4q\text{frk}p^2}}{2p} \\ \text{Adv}_{\text{FROST}_{2+}}^{\text{ts-suf-4}}(\mathcal{F}) &\leq \sqrt{q\text{Adv}_{\mathbb{G}}^{\text{omdl}}(\mathcal{C})} + \frac{q^2}{p} + \frac{q}{\sqrt{p}} \end{aligned}$$

with time $t_{\mathcal{C}} \leq t_{\mathcal{F}} + T(q)$ where $T(q)$ is the time required to simulate the game environment and compute $|CS| + 2q_{s1} + 1$ discrete logarithms. $\square$

4.1 Security with Distributed Key Generation

Our amendments to the pre-processing token generation and verification mechanism are orthogonal to key generation, allowing seamless integration with FROST2 when instantiated with secure distributed key generation (DKG) protocols such

as PedPoP [21,5]. This compatibility preserves the schemes' efficiency and security properties, as the token authentication do not interfere with DKG commitments or proofs.

FROST2 has established security records under DKG setting. Specifically, FROST2 combined with PedPoP is proven to be TS-UF-0 secure (and hence TS-UF-1 secure by extension) under the Schnorr-KoE and OMDL assumptions in the ROM [5]. More recently, Olaf [10] which extends FROST2 with SimplPedPoP, achieves TS-SUF-2 security.

Building on this foundation, FROST2+ under DKG can achieve TS-SUF-4 security under similar assumptions and models, as our token verification strengthens higher notions without relying on the fact that the proposed scheme is in the CKG settings.

To ensure completeness, we demonstrate that our proposed schemes maintain TS-SUF-4 security in the ROM when instantiated in the DKG setting using PedPoP (see Appendix C). Unlike Olaf, which employs a mixed forking approach, we leverage the segmentation features of the General Multi-Segment Forking Lemma to produce a more straightforward proof.

Theorem 2. *Our proposed FROST2+ signature scheme with ns signers and PedPoP is TS-SUF-4 secure in the random oracle model based on the assumption of one more discrete logarithm (OMDL) such that if there exists an algorithm $\mathcal{F}$ that breaks the TS-SUF-4 security with advantage*

$$\text{Adv}_{\text{FROST2+}}^{\text{ts-suf-4}}(\mathcal{F}) \leq \sqrt{q \text{Adv}_{\mathbb{G}}^{\text{omdl}}(\mathcal{C})} + \frac{q^2}{p} + \frac{q}{\sqrt{p}}$$

in time $t_{\mathcal{F}}$, then there exists an algorithm $\mathcal{C}$ that breaks OMDL in time $t_{\mathcal{C}} \leq t_{\mathcal{F}} + T(q)$ where $T(q)$ are time parameterized by $q = q_s + q_h + 2ns + 2$ the number of signing and hashing queries made.

Proof. The security proof is similar to that in Theorem 1 except the key generation simulation where instead of querying the OMDL oracles to implicitly define $f(Z)$, $\mathcal{C}$ defines $f_i(Z)$ for every honest signer. $\mathcal{C}$ also defines an additional $\text{RO}[\text{H}_{\text{kg}}]$ oracle to address the hash queries incurred by PedPoP:

$\text{RO}[\text{H}_{\text{kg}}]$: If $(M, \cdot) \in \text{HL}_{\text{kg}}$, return h . Else, return $h \xleftarrow{\$} \mathbb{Z}_p^*$ and parse M as $(X_{i,0}, X_{i,0}, \bar{R})$. Update $\text{HL}_{\text{kg}} \leftarrow \text{HL}_{\text{kg}} \cup \{(M, h)\}$.

After the forger $\mathcal{F}$ announcing the corrupted signers CS , let honest signers be the set $HS = \{\text{id}_i, \dots, \text{id}_{ns}\} \setminus CS$ and $k = |CS| < t$, $\mathcal{C}$ queries $A_0 = g^{a_0}$, $\{A_j = g^{a_j}\}_{j=1}^k \leftarrow \text{CHALL}$ and samples $\{a_j\}_{j=k+1}^{t-1} \xleftarrow{\$} \mathbb{Z}_p^*$. $\mathcal{C}$ selects $\{y_i\}_{\text{id}_i \in HS} \xleftarrow{\$} \mathbb{Z}_p^*$ to implicitly define the random polynomials $f_i(Z) = y_i(\sum_{j=0}^{t-1} a_j Z^j)$ for every honest signer $\text{id}_i \in HS$. Firstly, for every $\text{id}_i \in HS$, $\mathcal{C}$ queries $\bar{R}_i \leftarrow \text{CHALL}$, $h_i \leftarrow \text{RO}[\text{H}_{\text{kg}}](A_{i,0} = A_0^{y_i}, A_{i,0}, \bar{R}_i)$, $\bar{z}_i \leftarrow \text{DLOG}(\bar{R}_i A_{i,0}^{h_i})$ and broadcast $((\bar{R}_i, \bar{z}_i), C = (A_{i,0}, \dots, A_{i,t-1} = A_{t-1}^{y_i}))$ as the proof of possession (PoP). Secondly, $\mathcal{C}$ obtains $f'(\text{id}_j) \leftarrow \text{DLOG}(g^{f_i(\text{id}_j)/y_i})$ and sends $\bar{x}_{i,j} = y_i f'(\text{id}_j)$ to every corrupted signer

$\text{id}_j \in CS$. In return, $\mathcal{C}$ receives $\bar{x}_{j,i} = f_j(\text{id}_i)$ from $\text{id}_j \in CS$ for every $\text{id}_i \in HS$. So far, $\mathcal{C}$ issued $|CS| + |HS| + 1$ CHALL queries and $|HS| + |CS|$ DLOG queries.

As $\mathcal{C}$ simulates the distributed key generation perfectly, all PoP broadcast by honest signers are verified and every corrupted signer $\text{id}_i \in CS$ obtains their signing key share

$$x_i = f(\text{id}_i) = \sum_{j=1}^{\text{ns}} f_j(\text{id}_i) = \sum_{j=1}^{\text{ns}} \bar{x}_{j,i}$$

as expected, and the joint public key is

$$\tilde{X} = g^{a_0 \sum_{\text{id}_j \in HS} y_j + \sum_{\text{id}_i \in CS} a_{i,0}} = \prod_{\text{id}_j \in HS} A_{j,0} \prod_{\text{id}_i \in CS} A_{i,0}.$$

Subsequently, the oracle queries are answered in the same way as in Theorem 1. Finally, after rewinding the forger $\mathcal{F}$ to the primary index I_1 which is now the first hash queries recorded in HL_{kg} , $\mathcal{C}$ can extract the discrete logarithms for $A_{i,0} = A_0^{y_i}$ as $a_0 y_i$ for every signers $\text{id}_i \in HS$. For any $\text{id}_i \in HS$, with the knowledge of $(a_0, y_i, f_i(0) = a_0 y_i, \{a_j\}_{j=k+1}^{t-1}, \bar{x}_{i,j} = f_i(\text{id}_j))$, $\mathcal{C}$ can reconstruct the polynomial:

$$f_i(Z) = y_i \sum_{j=0}^{t-1} a_j Z^j$$

as in Theorem 1. Therefore, $\mathcal{C}$ recovered the $|CS| + 1$ discrete logarithms $\{a_j\}_{j=0}^{|CS|}$ queried during the key generation process.

The rest of the proof is identical and the success probability follows. $\square$

5 Extensions

5.1 Pre-Computations

Although the TS-SUF- i notions of Bellare et al. [5] assume a designated leader for collecting and distributing pre-processing tokens and signing requests to simplify security analysis, they explicitly cover scenarios in which corrupted signers take turns acting as the leader. Consequently, a leaderless design introduces no unmodeled attack vectors. In particular, attacks captured by TS-UF-3 and TS-UF-4, where a corrupted leader forging tokens or requests, are already included in the TS-SUF- i adversary model. When an honest signer initiates the protocol, the same corruption bounds apply, preserving the original security guarantees.

If our proposed FROST2+ adopts this leaderless approach, signers can collect and verify pre-processing tokens as in the original FROST [21] prior to the partial signing (PS) phase. This pre-verification step improves runtime performance, as shown in Table 2, without compromising security.

5.2 Batch Verification

While a signer from Bellare et al.’s ATS[FROST,DS] [7,5] can generate an offline signature on multiple tokens for batch verification, our FROST2+ generates an offline transcript on multiple tokens for the same purpose. Essentially, when a signer id_i has $n(\geq 2)$ tokens to be verified, during SPP, the signer computes:

$$\begin{aligned} \{r_{i,j}, s_{i,j}\}_{j=1}^n, v &\xleftarrow{\$} \mathbb{Z}_p^*, e = H_{\text{pp}}(\tilde{X}, X_i, \{R_{i,j}, S_{i,j}\}_{j=1}^n, V = g^v) \\ u = v + \text{st}_i \cdot \text{sk} \cdot e + \sum_{j=1}^n r_{i,j} e^{j+1} + \sum_{j=1}^n s_{i,j} e^{j+1+n}, \quad pp &\leftarrow (\{R_{i,j}, S_{i,j}\}_{j=1}^n, u, V) \end{aligned}$$

where the transcript size (u, V) remains constant. Subsequently, the verification is done by checking whether:

$$g^u \neq V X_i^{e'} \prod_{j=1}^n R_{i,j}^{e'^{j+1}} \prod_{j=1}^n S_{i,j}^{e'^{j+1+n}}$$

holds where $e' = H_{\text{pp}}(\tilde{X}, X_i, \{R_{i,j}, S_{i,j}\}_{j=1}^n, V)$.

Enabling this batch verification feature does not affect the TS-SUF-4 security, although the security reduction becomes looser. The general multi-segment forking lemma needs to additionally support multiple forkings as in Gennaro et al.’s multi-forking lemma [18]. More precisely, the forger needs to be rewound $2n + 1$ times for $2n$ tokens in a batch. If a tighter reduction is desirable, then one may need to rely on the Multi-Instance Reset Lemma [20].

5.3 Adaptive Security

Crites et al. [16] shows that key-unique threshold signature like FROST2 cannot achieve adaptive security in ROM for $\geq t/2$ corruption even assuming the hardness of interactive hard problems like (A)OMDL. Exceptions like FROST-Mask [9] which applies masking technique is not key-unique and can achieve full adaptive security under ROM. We see the potential in FROST2+ to achieve full adaptive security in ROM if the partial signature is masked as in related works [9,2,4] while retaining the TS-SUF-4 security in ROM.

6 Performance Evaluations

We present a comparison of the algorithmic complexity which focuses on scalar multiplications, and security of our schemes against FROST and its two-round variants in Table 2. While this analysis excludes hashing and point additions for simplicity, it highlights the core computational differences in scalar operations, with pre-processing contributions denoted in parentheses.

Next, we benchmarked the proposed FROST2+ against ZCash’s FROST implementation (GitHub commit 3ffc19d) [28] on a laptop with an Intel i9-14900HX

Table 2. Comparison of complexity and security for the proposed FROST+ against FROST and its variants with two signing rounds.

Scheme	Sign				Agg		Vf	Security			
	Round	$ pp $	$smul$	$\mathbb{G}$	$\mathbb{Z}_q$	$smul$		$smul$	Notion	Model	Corrupt
FROST [21]	2	2	$(2) + t$	2	1	t	2	TS-UF-1	ROM	Static	D
FROST [5]	2	2	$(2) + t$	3	1	0	2	TS-SUF-3	ROM	Static	C
FROST** [13,16]	2	2	$(2) + t$	2	1	t	2	adp-TS-UF	AGM	Adaptive	C
ATS[TS, DS]* [5]	2	4	$(3) + 3t$	4	2	0	2	TS-SUF-4	ROM	Static	C
FROST2 [14]	2	2	$(2)+1$	2	1	1	2	TS-UF-1	ROM	Static	D
FROST2 [5]	2	2	$(2)+1$	3	1	0	2	TS-SUF-2	ROM	Static	C
ROAST [25]	2	2	$(2)+1$	2	1	1	2	TS-SUF-2	ROM	Static	C
Olaf [10]	2	2	$(2)+1$	2	1	1	2	TS-SUF-2	ROM	Static	D
Arctic [22]	2	1	$(1) + t^2 + t$	1	1	t	2	UF	ROM	Static	C,D
ms-FROST [2]	2	2	$(2)+1$	3	1	0	2	adp-TS-UF-4	AGM	Adaptive	C
FaFROST [4]	2	2	$(2)+1$	3	1	0	2	adp-TS-UF	AGM	Adaptive	C
FROST-Mask [9]	2	2	$(2)+1$	3	1	1	2	adp-UF	ROM	Adaptive	C
FROST2** [16]	2	2	$(2) + 1$	3	1	0	2	adp-TS-UF	AGM	Adaptive	C
FROST3** [16]	2	2	$(2) + 1$	2	1	1	2	adp-TS-UF	AGM	Adaptive	C
FROST2+	2	4	$(3) + 4t + 1$	4	2	0	2	TS-SUF-4	ROM	Static	C,D
FROST2+ ^s	2	4	$(3 + 4t) + 1$	4	2	0	2	TS-SUF-4	ROM	Static	C,D

Note: * Assume DS is the Schnorr signature scheme, ^s: with pre-computations, n : total signers, ** Maximum of $t/2$ adaptive corruptions if in ROM, $smul$: scalar multiplication, t : threshold, (num) : op. in pre-processing round, Kg: key generation, C: centralized, D: decentralized $\mathbb{G}, \mathbb{Z}_q$: numbers of group and field element sent per signer

(2.20 GHz), 32 GB RAM, running Windows 11 Home Single Edition. To support our (and FROST2's) CompPar algorithm, which differs from the original FROST design, we modified the functions `binding_factor_preimages` and `compute_group_commitment` in `frost-core/src/lib.rs`.

We evaluated FROST, FROST2 and FROST2+ using the Ed25519 curve across various predefined (t, n) parameter sets. For each configuration, we executed 100 rounds and recorded the average time per signature generation. Each timing experiment was conducted twice: once with pre-computations enabled and once with them disabled. When pre-computations were enabled, the timings for token verification and the pre-processing round were excluded from the signature generation time; when disabled, they were included. The full numerical results are provided in Table 3 (Appendix D), and Figure 3 visualizes the signatures-per-second for all schemes.

The evaluation results shown in Table 3 and Figure 3 not only confirm but also extend the complexity estimates by revealing the impact of hashing and point additions. When pre-computations are enabled, FROST2+ achieves throughput nearly identical to FROST2 across all group sizes, demonstrating that its additional features come with negligible overhead. Specifically, at $n = 200$ signers, FROST2+ reaches 913 signatures/sec, marginally surpassing FROST2

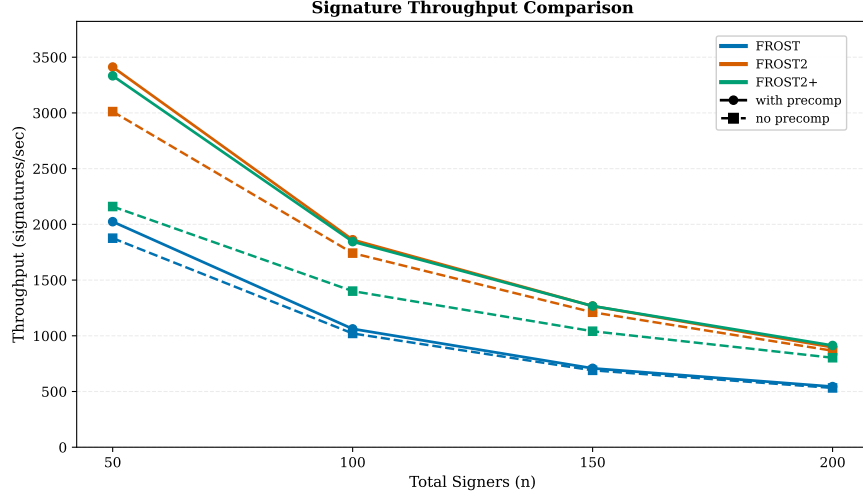


Fig. 3. Signatures generated per seconds for FROST, FROST2, FROST2+.

(897 sig/s) and outperforming FROST by a factor of $1.7\times$ (544 sig/s). For smaller group sizes, FROST2+ remains comparable to FROST2, occasionally within 2–3% difference, indicating that its optimizations scale effectively. The performance gap between the FROST2+ and FROST widens as the group size increases. At $n = 50$, FROST2+ is approximately $1.65\times$ faster than FROST with pre-computation while at $n = 200$, this ratio grows to $1.68\times$, confirming the better scalability of the underlying algorithms in FROST2 and FROST2+.

When pre-computations are disabled, FROST2+ exhibits a noticeable performance degradation compared to FROST2 (e.g., 2160 vs. 3012 sig/s at $n = 50$). This suggests that the optimizations in FROST2+ rely heavily on the pre-computations to achieve competitive throughput, and without it the additional operations from authenticating tokens impose a measurable cost. Nonetheless, even without pre-computations, FROST2+ outperforms FROST for all tested n , demonstrating the efficiency gained from the lighter-weight CompPar algorithm.

7 Conclusion

We presented an enhanced FROST2 scheme, namely, FROST2+ with proven TS-SUF-4 security under OMDL assumption in the ROM with static corruption and (de)centralized key generation settings. FROST2+ is the only ROM-based TS-SUF-4 scheme to date apart from Bellare et al.’s ATS[FROST,DS] where the primitive FROST used there is 30% to 40% slower than FROST2+ for a signing group with threshold ($t = 140, n = 200$).

References

1. Arch Network: Arch’s signature scheme model (frost + roast). <https://docs.arch.network/learn/fundamentals/arch-signature-scheme-model-frost-roast> (2024), documentation page, last updated 2024; accessed Nov 18, 2025
2. Bacho, R., Chen, Y., Loss, J., Tessaro, S., Zhu, C.: Adaptively secure partially non-interactive threshold schnorr signatures in the AGM. Cryptology ePrint Archive, Paper 2025/1953 (2025), <https://eprint.iacr.org/2025/1953>
3. Bacho, R., Das, S., Loss, J., Ren, L.: Adaptively secure three-round threshold schnorr signatures from ddh. In: Advances in Cryptology – CRYPTO 2025: 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17–21, 2025, Proceedings, Part VI. p. 390–422. Springer-Verlag, Berlin, Heidelberg (2025). https://doi.org/10.1007/978-3-032-01887-8_13, https://doi.org/10.1007/978-3-032-01887-8_13
4. Baecker, R., Gerhart, P., Calsi, D.L., Russo, L., Schröder, D., Yerukhimovich, A.: Fully adaptive FROST in the algebraic group model from falsifiable assumptions. Cryptology ePrint Archive, Paper 2025/1950 (2025), <https://eprint.iacr.org/2025/1950>
5. Bellare, M., Crites, E., Komlo, C., Maller, M., Tessaro, S., Zhu, C.: Better than advertised security for non-interactive threshold signatures. In: Dodis, Y., Shrimpton, T. (eds.) Advances in Cryptology – CRYPTO 2022. pp. 517–550. Springer Nature Switzerland, Cham (2022)
6. Bellare, M., Neven, G.: Multi-signatures in the plain public-key model and a general forking lemma. In: Proceedings of the 13th ACM Conference on Computer and Communications Security. p. 390–399. CCS ’06, Association for Computing Machinery, New York, NY, USA (2006). <https://doi.org/10.1145/1180405.1180453>, <https://doi.org/10.1145/1180405.1180453>
7. Bellare, M., Tessaro, S., Zhu, C.: Stronger security for non-interactive threshold signatures: BLS and FROST. Cryptology ePrint Archive, Paper 2022/833 (2022), <https://eprint.iacr.org/2022/833>
8. Brandão, L.T.A.N., Peralta, R.: NIST First Call for Multi-Party Threshold Schemes (NISTIR 8214C, draft). NIST Interagency Report (NISTIR) – Draft 8214C, National Institute of Standards and Technology (NIST) (2023), <https://csrc.nist.gov/publications/detail/nistir/8214c/draft>
9. Chen, Y.: Round-efficient adaptively secure threshold signatures with rewinding. IACR Communications in Cryptology **2**(2) (2025). <https://doi.org/10.62056/ah893z10k>
10. Chu, H., Gerhart, P., Ruffing, T., Schröder, D.: Practical schnorr threshold signatures without the algebraic group model. In: Handschuh, H., Lysyanskaya, A. (eds.) Advances in Cryptology – CRYPTO 2023. pp. 743–773. Springer Nature Switzerland, Cham (2023)
11. Connolly, D., Komlo, C., Goldberg, I., Wood, C.A.: Rfc 9591 the flexible round-optimized schnorr threshold (frost) protocol for two-round schnorr signatures (2024)
12. Cremers, C., Peltonen, A., Zhao, M.: An extended hierarchy of security notions for threshold signature schemes and automated analysis of protocols that use them. Cryptology ePrint Archive, Paper 2024/1920 (2024), <https://eprint.iacr.org/2024/1920>
13. Crites, E., Katz, J., Komlo, C., Tessaro, S., Zhu, C.: On the adaptive security of frost. In: Tauman Kalai, Y., Kamara, S.F. (eds.) Advances in Cryptology – CRYPTO 2025. pp. 480–511. Springer Nature Switzerland, Cham (2025)

14. Crites, E., Komlo, C., Maller, M.: How to prove schnorr assuming schnorr: Security of multi- and threshold signatures. Cryptology ePrint Archive, Paper 2021/1375 (2021), <https://eprint.iacr.org/2021/1375>
15. Crites, E., Komlo, C., Maller, M.: Fully adaptive schnorr threshold signatures. In: Advances in Cryptology – CRYPTO 2023: 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20–24, 2023, Proceedings, Part I. p. 678–709. Springer-Verlag, Berlin, Heidelberg (2023). https://doi.org/10.1007/978-3-031-38557-5_22, https://doi.org/10.1007/978-3-031-38557-5_22
16. Crites, E., Komlo, C., Maller, M.: On the adaptive security of key-unique threshold signatures. Cryptology ePrint Archive, Paper 2025/943 (2025), <https://eprint.iacr.org/2025/943>
17. Crites, E., Stewart, A.: A plausible attack on the adaptive security of threshold schnorr signatures. In: Tauman Kalai, Y., Kamara, S.F. (eds.) Advances in Cryptology – CRYPTO 2025. pp. 457–479. Springer Nature Switzerland, Cham (2025)
18. Gennaro, R., Leigh, D., Sundaram, R., Yerazunis, W.: Batching schnorr identification scheme with applications to privacy-preserving authorization and low-bandwidth communication devices. In: Lee, P.J. (ed.) Advances in Cryptology – ASIACRYPT 2004 (Lecture Notes in Computer Science, vol. 3329). Springer, Berlin, Heidelberg (2004). https://doi.org/10.1007/978-3-540-30539-2_20
19. Harman, S., Shishmarev, M., Nash, T., Lennon, D., Janse van Vuuren, S.: Chainflip protocol whitepaper. Tech. rep., Auxilio Ventures (2022), <https://www.auxilioventures.com/wp-content/uploads/2022/08/Chainflip-Whitepaper-May-2020.pdf>
20. Kiltz, E., Masny, D., Pan, J.: Optimal security proofs for signatures from identification schemes. In: Robshaw, M., Katz, J. (eds.) Advances in Cryptology – CRYPTO 2016. pp. 33–61. Springer Berlin Heidelberg, Berlin, Heidelberg (2016)
21. Komlo, C., Goldberg, I.: Frost: Flexible round-optimized schnorr threshold signatures. In: Dunkelman, O., Jacobson, Jr., M.J., O’Flynn, C. (eds.) Selected Areas in Cryptography. pp. 34–65. Springer International Publishing, Cham (2021)
22. Komlo, C., Goldberg, I.: Arctic: Lightweight and stateless threshold schnorr signatures. In: Jager, T., Pan, J. (eds.) Public-Key Cryptography – PKC 2025. pp. 234–267. Springer Nature Switzerland, Cham (2025)
23. Mccorry, P., Bakshi, S., Dembo, A., Miller, A.: sbtc: Design of a trustless two-way peg for bitcoin. Tech. rep., Trustless Computing Foundation (2023), <https://sbtc.tech/sbtc.pdf>
24. Niot, G., Reichle, M., Takemure, K.: Adaptively-secure three-round threshold schnorr from DL. Cryptology ePrint Archive, Paper 2025/1941 (2025), <https://eprint.iacr.org/2025/1941>
25. Ruffing, T., Ronge, V., Jin, E., Schneider-Bensch, J., Schröder, D.: Roast: Robust asynchronous schnorr threshold signatures. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. p. 2551–2564. CCS ’22, Association for Computing Machinery, New York, NY, USA (2022). <https://doi.org/10.1145/3548606.3560583>, <https://doi.org/10.1145/3548606.3560583>
26. Safe Research: safe-frost. <https://github.com/safe-research/safe-frost> (2024), gitHub repository; accessed Nov 18, 2025
27. Schnorr, C.P.: Efficient signature generation by smart cards. Journal of Cryptology 4, 161–174 (1991). <https://doi.org/10.1007/BF00196725>
28. Zcash Foundation: Speeding up frost with multi-scalar multiplication, <https://zfnf.org/speeding-up-frost-with-multi-scalar-multiplication/>

A Attacks on FROST2

A.1 TS-UF-3 Attack

In Bellare et al.'s work [7,5], they provide a security proof of TS-SUF-2 and also a TS-UF-3 attack for FROST2. This indicates that the best possible security level FROST2 can achieve is only TS-SUF-2. We recall Bellare et al.'s TS-UF-3 attack in the setting below:

- Total signers are $\{P_1, P_2, P_3, P_4\}$.
- Signers $\{P_3, P_4\}$ corrupted.
- A signing request on signer set $lr.\mathcal{S} = \{1, 2, 3\}$ for a message $lr.msg = M$.
- The pre-processing token $(R_1, S_1), (R_2, S_2)$ are known to the corrupted signers before they generate theirs.
- First signer's partial signature $z_1 = r_1 + ds_1 + c\Delta_{1,\mathcal{S}}(0)x_1$ can be obtained.
- Expected combined signature is $z = \text{dlog}(\tilde{R}) + cx = \sum_{i \in \mathcal{S}} r_i + ds_i + c\Delta_{i,\mathcal{S}}(0)x_i$

The attack procedure is as follows:

1. P_3 generates its pre-processing token as

$$R_3 = R_1^{\frac{\Delta_{1,\mathcal{S}'(0)}}{\Delta_{1,\mathcal{S}(0)}} - 1} R_2^{-1}, S_3 = S_1^{\frac{\Delta_{1,\mathcal{S}'(0)}}{\Delta_{1,\mathcal{S}(0)}} - 1} S_2^{-1}.$$

where $\mathcal{S}' = \{1, 3, 4\}$. The resulting group pre-processing token will be:

$$\tilde{R} = R_1^{\frac{\Delta_{1,\mathcal{S}'(0)}}{\Delta_{1,\mathcal{S}(0)}}} S_1^{d \frac{\Delta_{1,\mathcal{S}'(0)}}{\Delta_{1,\mathcal{S}(0)}}} = \prod_{i \in \mathcal{S}} R_i S_i^d$$

where $\text{dlog}(\tilde{R}) = r_1 \frac{\Delta_{1,\mathcal{S}'(0)}}{\Delta_{1,\mathcal{S}(0)}} + ds_1 \frac{\Delta_{1,\mathcal{S}'(0)}}{\Delta_{1,\mathcal{S}(0)}}$ and $d = H_{\text{non}}(\tilde{X}, lr)$.

2. P_3 computes $z'_1 = r_1 \frac{\Delta_{1,\mathcal{S}'(0)}}{\Delta_{1,\mathcal{S}(0)}} + ds_1 \frac{\Delta_{1,\mathcal{S}'(0)}}{\Delta_{1,\mathcal{S}(0)}} + c\Delta_{1,\mathcal{S}'}(0)x_1 = \frac{\Delta_{1,\mathcal{S}'(0)}}{\Delta_{1,\mathcal{S}(0)}} z_1$
3. P_3 and P_4 forge a valid combined signature for M under $lr.\mathcal{S}$ as

$$z' = \text{dlog}(\tilde{R}) + cx = z'_1 + c\Delta_{3,\mathcal{S}'}(0)x_3 + c\Delta_{4,\mathcal{S}'}(0)x_4.$$

This is a valid TS-UF-3 attack because $S_2 = \{1\}$ but unequal to $S_3 = \{1, 2\}$. We note that this attack does not work on our schemes because the attacker's pre-processing token (R_3, S_3) is a combination of the honest signers' pre-processing token whose discrete logarithms are not known. Consequently, a valid authentication transcript (u, V) for (R_3, S_3) cannot be generated and the honest signers abort during PSIGNO.

A.2 TS-UF-4 Attack

In Bellare et al.'s work [7,5], they demonstrate a TS-UF-4 attack for FROST but not FROST2. Here, we mount the attack on FROST2 as follows:

- Total signers are $\{P_5, P_{10}, P_{11}, P_{15}, P_{20}\}$.

- Signers $\{P_5, P_{10}\}$ corrupted.
- A signing request on signer set $lr.\mathcal{S} = \{11, 15, 20\}$ for a message $lr.\text{msg} = M$.
- The pre-processing token $lr.\text{PP} = \{(R_{11}, S_{11}), (R_{15} = g^{r_{15}}, S_{15} = g^{s_{15}}), (R_{20} = g^{r_{20}}, S_{20} = g^{s_{20}})\}$ and $r_{15}, s_{15}, r_{20}, s_{20} \in \mathbb{Z}_p^*$ are known to the corrupted signers before they generate theirs.
- First signer's partial signature $z_{11} = r_{11} + d_{11}s_{11} + c\Delta_{11,\mathcal{S}}(0)x_{11}$ can be obtained.
- Expected combined signature is $z = \text{dlog}(\tilde{R}) + cx = \sum_{i \in \mathcal{S}} r_i + ds_i + c\Delta_{i,\mathcal{S}}(0)x_i$

The attack procedure is as follows:

1. Let $\mathcal{S}' = \{5, 10, 11\}$, the Lagrange coefficients are the same for:

$$\Delta_{11,lr.\mathcal{S}} = \frac{15 \cdot 20}{(15 - 11)(20 - 11)} = \frac{25}{3} = \frac{5 \cdot 10}{(5 - 11)(10 - 11)} = \Delta_{11,\mathcal{S}'}$$

The resulting group pre-processing token will be:

$$\tilde{R} = R_{11}S_{11}^d g^{r_{15} + ds_{15} + r_{20} + ds_{20}} = \prod_{i \in \mathcal{S}} R_i S_i^d$$

where $\text{dlog}(\tilde{R}) = r_{11} + ds_{11} + r_{15} + ds_{15} + r_{20} + ds_{20}$ and $d = \text{H}_{\text{non}}(\tilde{X}, lr)$.

2. P_5 and P_{10} forge a valid combined signature for M under $lr.\mathcal{S}$ as

$$z' = \text{dlog}(\tilde{R}) + cx = z_{11} + r_{15} + ds_{15} + r_{20} + ds_{20} + c\Delta_{5,\mathcal{S}'}(0)x_5 + c\Delta_{10,\mathcal{S}'}(0)x_{10}.$$

This is a valid TS-UF-4 attack because $S_2 = \{11\} = S_3$ but unequal to $S_4 = \{11, 15, 20\}$. We note that this attack does not work on our schemes for the similar reason in Section A.1. Although the attacker knows the discrete logarithms for the pre-processing tokens $(R_{15}, S_{15}), (R_{20}, S_{20})$, it does not know the honest signers' shared signing keys which are required to generate the authentication transcripts $(u_{15}, V_{15}), (u_{20}, V_{20})$ for the tokens. Therefore, the honest signer will abort during partial signing because of invalid $(u_{15}, V_{15}), (u_{20}, V_{20})$.

B General Forking Lemma

We recall the general forking lemma of Bellare and Neven [6] in Algorithm 2.

Lemma 2 (General Forking Lemma). *Let acc be the probability that $\mathcal{A}$ returns a non-zero index I in its first execution in GFork . Let frk be the probability that $\text{GFork}(x)$ outputs $(1, \sigma, \sigma')$.*

$$\text{frk} \geq \text{acc} \cdot \left(\frac{\text{acc}}{q} - \frac{1}{|\mathbb{H}|} \right)$$

where q is the number of hash queries and $\mathbb{H}$ is the range of the random oracle.

Algorithm 2 General Forking Algorithm $\text{GFork}_{\mathcal{A}}(x)$

```

1: Pick coins  $\rho$  for  $\mathcal{A}$  at random
2:  $\{h_1, \dots, h_q\} \xleftarrow{\$} \mathbb{H}^q$ 
3:  $Q \leftarrow \{h_1, \dots, h_q\}$ 
4:  $(I, \sigma) \leftarrow \mathcal{A}(x, Q; \rho)$ 
5: if  $I = 0$  then
6:   return  $(0, \perp, \perp)$ 
7: end if
8:  $h'_1, \dots, h'_q \xleftarrow{\$} \mathbb{H}^{q-I+1}$ 
9:  $Q' \leftarrow \{h_1, \dots, h_{I-1}\} \cup \{h'_1, \dots, h'_q\}$ 
10:  $(I', \sigma') \leftarrow \mathcal{A}(x, Q'; \rho)$ 
11: if  $(I = I' \text{ and } h_I \neq h'_I)$  then
12:   return  $(1, \sigma, \sigma')$ 
13: else
14:   return  $(0, \perp, \perp)$ 
15: end if

```

C PedPoP

We recall PedPoP key generation [21, 5] in Figure 4.

D Detailed Benchmark Results

The detailed benchmark data is in Table 3.

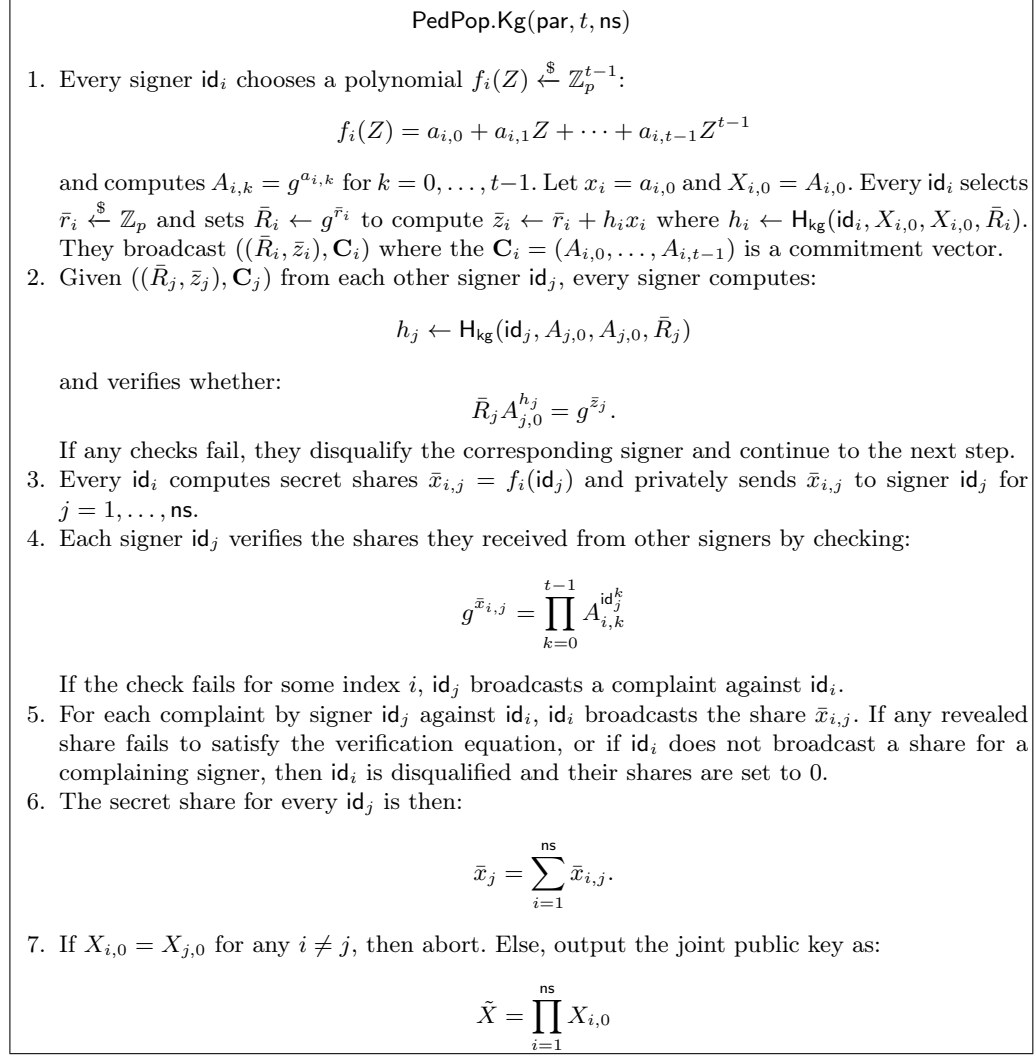
**Fig. 4.** The PedPoP algorithm [21,5].

Table 3. Performance comparison of threshold signature schemes (excluding DKG setup). (t, n) denotes the threshold and total signers.

Scheme	Pre-comp	(t, n)	Signature Gen. (ms)	Throughput (sig/sec)	Security
FROST	✓	(35, 50)	0.494	2,024	TS-SUF-3
		(70, 100)	0.941	1,062	
		(105, 150)	1.412	708	
		(140, 200)	1.838	544	
	✗	(35, 50)	0.533	1,876	
		(70, 100)	0.979	1,021	
		(105, 150)	1.449	690	
		(140, 200)	1.879	532	
FROST2	✓	(35, 50)	0.293	3,412	TS-SUF-2
		(70, 100)	0.537	1,862	
		(105, 150)	0.789	1,267	
		(140, 200)	1.115	897	
	✗	(35, 50)	0.332	3,012	
		(70, 100)	0.574	1,742	
		(105, 150)	0.826	1,211	
		(140, 200)	1.155	866	
FROST2+	✓	(35, 50)	0.300	3,333	TS-SUF-4
		(70, 100)	0.542	1,845	
		(105, 150)	0.789	1,267	
		(140, 200)	1.095	913	
	✗	(35, 50)	0.463	2,160	
		(10, 100)	0.714	1,401	
		(105, 150)	0.961	1,041	
		(140, 200)	1.245	803	